

Singleton Design Pattern Real-Time Example – Caching using C#

In this article, I will discuss **How to Implement Singleton Design Pattern in Real-Time Example Caching using C#**. Please read our previous article discussing [Singleton Design Pattern Real-Time Example Logging](#) using ASP.NET MVC Application.

Singleton Design Pattern Real-Time Example – Caching

Let us see how we can implement the Singleton Design Pattern Real-Time Example Caching using C#. So, basically, we will create the class as Singleton, and the class will provide the basic Caching functionality. So, by using the Singleton instance, we can add elements to the cache, update the elements in the cache, fetch the elements from the cache, delete a particular element from the cache, and delete all the elements from the cache.

In our [Singleton vs Static Class](#) article, we have discussed that the Singleton class can be inherited from another class, whereas the Static class is never inherited from another class. So, let us see how the Singleton class is inherited from another class in this Caching Example.

Step1: Create the Cache Interface

First, create a class file named **IMyCache.cs** and copy and paste the following code. This interface declares the operations that are required for Caching. Here, you can see we are using the object as the parameter for both keys and values to store any data in the Cache.

```
namespace SingletonDesignPatternRealTimeExample
{
    //Declaring the Interface for Cache Operations
    //This Interface is going to be Implemented by Singleton Class
    public interface IMyCache
    {
        bool Add(object key, object value);
        bool AddOrUpdate(object key, object value);
        object Get(object key);
        bool Remove(object key);
        void Clear();
    }
}
```

Step2: Creating the Singleton Class

Now, we need to create the Singleton Class by inheriting from the above IMyCache interface and provide implementation to all five interface methods. So, create a class file with the name **SingletonCache.cs** and copy and paste the following code.

```
using System;
using System.Collections.Concurrent;
namespace SingletonDesignPatternRealTimeExample
{
    public sealed class SingletonCache : IMyCache
    {
        //Shared resource that needs protection in a Multithreaded
        Environment
        //ConcurrentDictionary Collection is Thread-Safe
```

```

        private ConcurrentDictionary<object, object> concurrentDictionary =
new ConcurrentDictionary<object, object>();
        //This variable is going to store the Singleton Instance
        //Initializing the Variable at the time of class start-up
        //and make it ready to be used in the Future
        private static readonly SingletonCache singletonInstance = new
SingletonCache();
        //The following Static Method is going to return the Singleton
Instance
        //This is Thread-Safe as it uses Eager Loading
        public static SingletonCache GetInstance()
        {
            return singletonInstance;
        }
        //Constructor needs to be Private in order to restrict
        //class instantiation from Outside of this class
        private SingletonCache()
        {
            Console.WriteLine("SingletonCache Instance Created");
        }
        //The following methods can be accessed from outside of the class
by using the Singleton Instance

        //This method is used to add a Key-Value Pair into the Cache
        public bool Add(object key, object value)
        {
            return concurrentDictionary.TryAdd(key, value);
        }
        //This method is used to add or update a Key-Value Pair into the
Cache
        //If the Key is not available then add the key-value pair
        //If the Key is already added, then update the value of that key
        public bool AddOrUpdate(object key, object value)
        {
            if (concurrentDictionary.ContainsKey(key))
            {
                concurrentDictionary.TryRemove(key, out object
RemovedValue);
            }
            return concurrentDictionary.TryAdd(key, value);
        }
        //This method is used to return the value of the given key if
present in the cache
        //Else return null
        public object Get(object key)
        {
            if (concurrentDictionary.ContainsKey(key))
            {
                return concurrentDictionary[key];
            }
            return null;
        }
        //This method is used to remove the given key from the cache
        //If removed return true else return false

```

```

        public bool Remove(object key)
        {
            return concurrentDictionary.TryRemove(key, out object value);
        }
        //This method is used to remove the given key from the cache
        //If removed return true else return false
        public void Clear()
        {
            // Removes all keys and values from the ConcurrentDictionary
            i.e. from the Cache
            concurrentDictionary.Clear();
        }
    }
}

```

Code Explanations:

1. In the above SingletonCache class, we have implemented the Singleton Design Pattern using Eager Loading to avoid Thread-Safety Issues while running the application in a Multithreaded Environment.
2. Here, we use the ConcurrentDictionary collection to store the data using key and value pairs. This is going to act like a Cache. We use the ConcurrentDictionary collection instead of the Generic Dictionary because the ConcurrentDictionary collection is thread-safe by default, so we will not get any thread-safety issues when working with a multithread environment.
3. The Add method expects the Key and Value as a parameter. Then, it adds the key and values into the ConcurrentDictionary collection if the key does not already exist in the collection. If the key exists, it will not add; in that case, it will return False.
4. The AddOrUpdate method also expects the Key and Value as a parameter. Then, it adds the key and values into the ConcurrentDictionary collection if the key does not already exist in the collection. It will update the existing key value with the received new value if the key exists.
5. The Get method will return the value based on the given key. If the key does not exist in the collection, it will return null.
6. The Remove method will remove the key from the Cache or Collection and return true. If the key does not exist, then it will return false.
7. The Clear method will remove all the keys and values from the collection.

Step3: Using Singleton Instance and Caching in Client Code

In our example, the Program class is going to be the Client Code. Let us modify the Main method of the Program class to use the Singleton Instance to understand how we can perform the Caching using the Singleton Class.

```

using System;
namespace SingletonDesignPatternRealTimeExample
{
    class Program
    {
        static void Main(string[] args)
        {
            //Get the SingletonCache Instance
            SingletonCache cache = SingletonCache.GetInstance();
            //Adding Keys and Values into the Cache using Add and
            AddOrUpdate Method
            Console.WriteLine("Adding Keys and Values into Cache");
            Console.WriteLine($" Adding Id in Cache: {cache.Add("Id",
            1001)}}");
            Console.WriteLine($" Adding Name in Cache: {cache.Add("Name",
            "Pranaya)}}");
        }
    }
}

```

```

        Console.WriteLine($" Adding Same Id Key in Cache using Add:
{cache.Add("Id", 2001)}");
        Console.WriteLine($" Adding Same Id Key in Cache using
AddOrUpdate: {cache.AddOrUpdate("Id", 2001)}");
        //Accessing values from Cache using Keys using the Get Method
by passing the Keys
        Console.WriteLine("\nFetching Values from Cache");
        Console.WriteLine($" Fetching Id From Cache:
{cache.Get("Id")}");
        Console.WriteLine($" Fetching Name From Cache:
{cache.Get("Name")}");
        //Removing Elements from Cache using Remove Method by passing
the given keys
        Console.WriteLine("\nRemoving Values from Cache");
        Console.WriteLine($" Remove Id: {cache.Remove("Id")}");
        Console.WriteLine($" Accessing Id From Cache:
{cache.Get("Id")}");
        //Removing All Elements from the Cache using the Clear Method
cache.Clear();
        Console.WriteLine("\nClearing All Keys and Values");
        Console.WriteLine($" Fetching Name From Cache:
{cache.Get("Name")}");
        Console.ReadKey();
    }
}

```

Output:

```

SingletonCache Instance Created
Adding Keys and Values into Cache
  Adding Id in Cache: True
  Adding Name in Cache: True
  Adding Same Id Key in Cache using Add: False
  Adding Same Id Key in Cache using AddOrUpdate: True

Fetching Values from Cache
  Fetching Id From Cache: 2001
  Fetching Name From Cache: Pranaya

Removing Values from Cache
  Remove Id: True
  Accessing Id From Cache:

Clearing All Keys and Values
  Fetching Name From Cache:

```

In the next article, I will discuss [Structural Design Patterns in C#](#) with Examples. In this article, I explain How to Implement Singleton Design Pattern Real-Time Example Caching using C#. I hope you enjoy this Singleton Design Pattern Real-Time Example Caching article.